



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Constituent of Symbiosis International (Deemed University), Pune

Name: Devansh Singh
PRN: 25070122072
Batch: A-2
Division: A

Subject: Programming with Java

ASSIGNMENT No : 10

TITLE : Write a Java program to demonstrate the use of abstract classes and abstract methods.

Problem Statement

Write a Java program to demonstrate the use of abstract classes and abstract methods.

Learning Objectives

To implement abstraction using abstract classes and abstract methods.

Theory

Abstract classes provide partial implementation and abstract methods must be implemented by subclasses. Abstraction hides implementation details while exposing essential functionality.

Output Screenshots

```
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp10/Main.java
Drawing a circle.
This is a shape
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$
```

Exercise

Q1. Create an abstract Payment class and implement Credit Card and UPI payment classes.

```
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp10/Exercise/Ex1.java
Payment Method
Paid Rs.2500.0 using Credit Card.

Payment Method
Paid Rs.1500.0 using UPI.
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$
```

Q2. Create an abstract FoodOrder class with an abstract method calculate Bill(). Implement two subclasses, DineInOrder and TakeAwayOrder, to calculate and display the total bill based on the order type.

```
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp10/Exercise/Ex2.java
Dine-In Bill : Rs.1100.0
Take-Away Bill : Rs.1050.0
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$
```

Conclusion

This assignment helped in understanding the concepts of abstract classes and abstract methods in Java. It demonstrated how abstraction hides implementation details while exposing only the essential functionality, allowing subclasses to provide their own implementations of abstract methods. The implementation improved my understanding of object-oriented programming principles such as abstraction, inheritance, and polymorphism, resulting in more flexible and maintainable code. These concepts are widely used in developing desktop applications, web applications, Android applications, enterprise software, banking systems, and other real-world Java applications that require reusable, modular, and scalable software design.